

SOFTWARE ENGINEERING

Prof. Dr. Christoph Andriessens
christoph.andriessens@hs-weingarten.de



ENTWURF

Architektur umsetzen

Themen in dieser Vorlesung



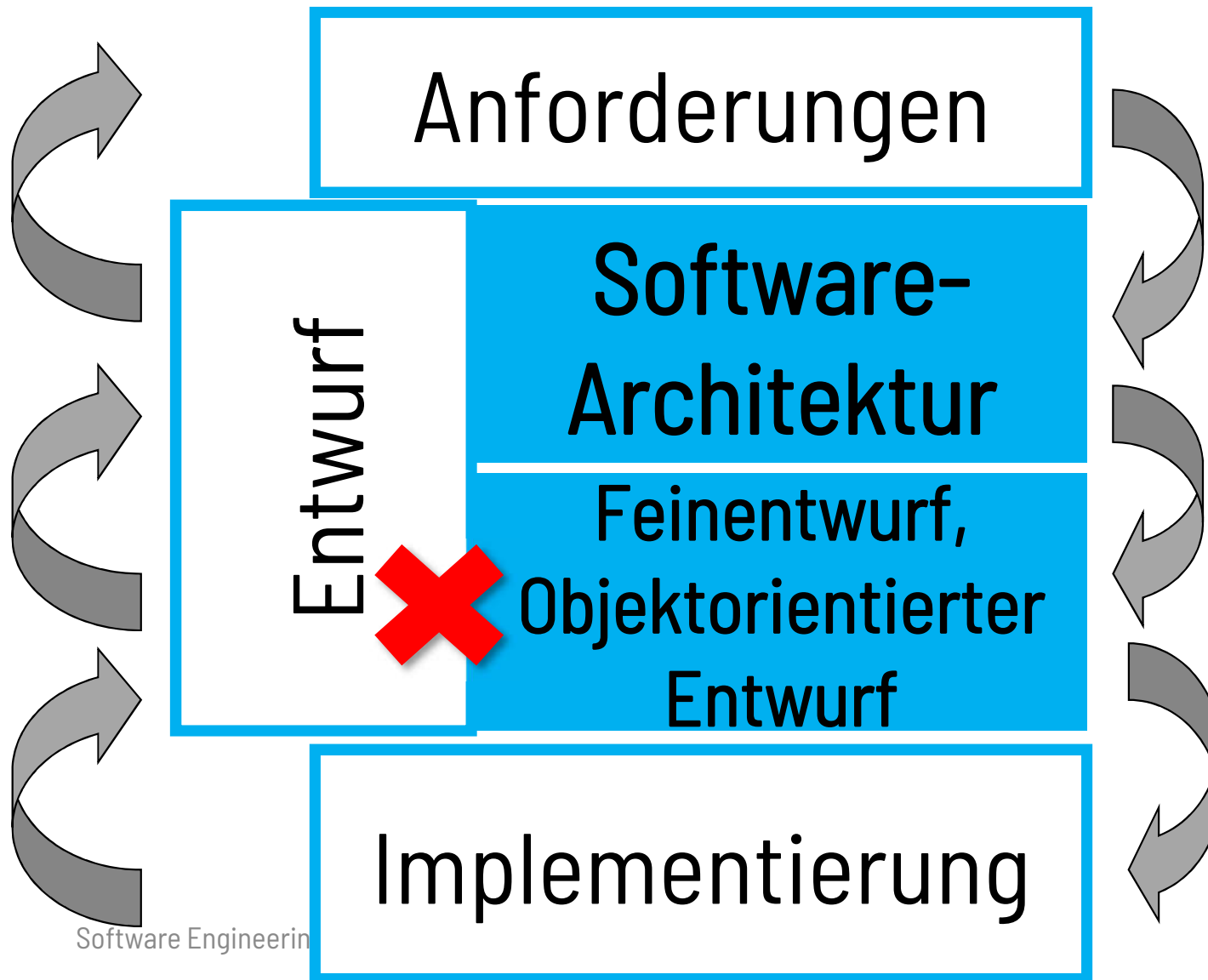
Lernziele

- Wesentliche Prinzipien im objektorientierte Entwurf kennen und anwenden können
- Heuristiken für den objektorientierten Entwurf kennenlernen



Unser Thema:

Entwurf innerhalb der Architektur



Beziehung von Architektur und Entwurf (Design)

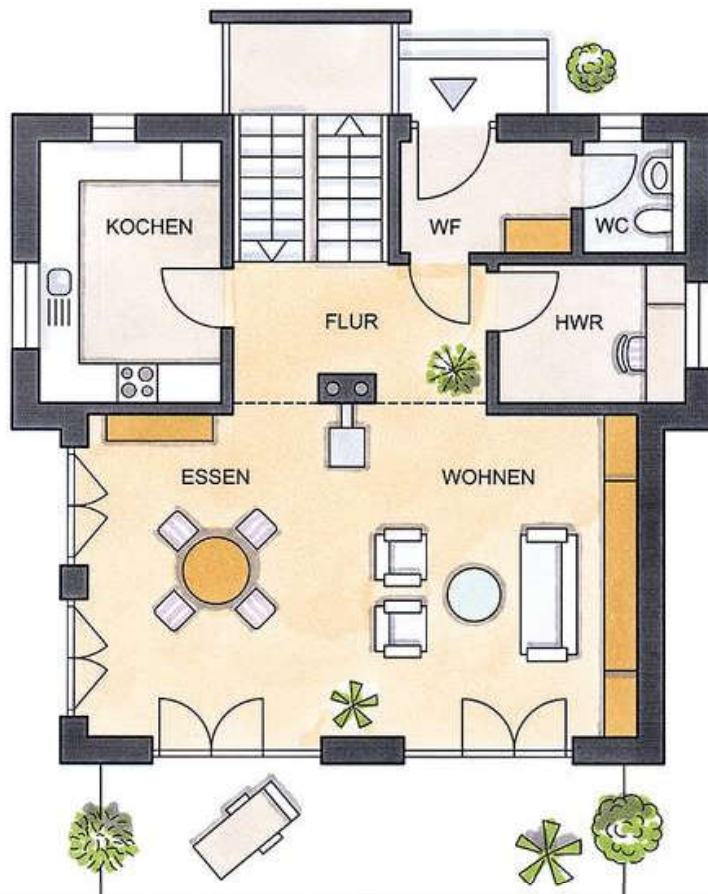
„All architecture is design, but not all design is architecture.“

Grady Booch, IBM Fellow, Chief Scientist SE

- Architektur drückt **signifikante Entwurfsentscheidungen** für ein System aus
- Signifikant = „schwer und teuer zu ändern“
- Architektur ist auch in Technik gefaßte **Unternehmensstrategie**: Welchen Wert erzeugt ein Unternehmen für wen und mit welchen Mitteln



Architektur? Entwurf?



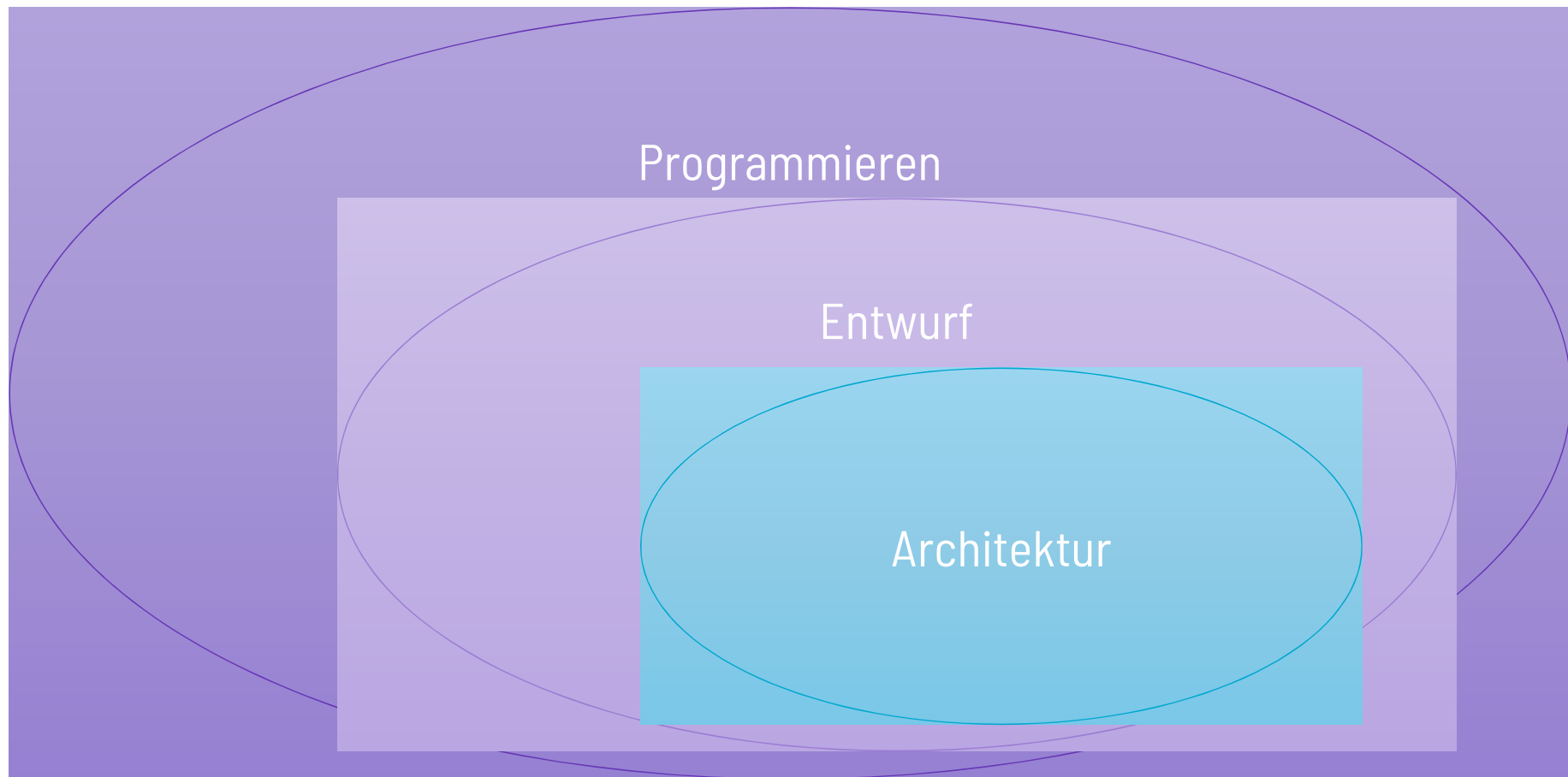
Nur Architektur: Plan des Objekts mit grundlegenden Strukturen



Entwurf: Vollständige Ausgestaltung des Objektes mit Strukturen und Details (Material, Farben, ...)



Programmieren, Entwurf, Architektur? Zusammenhänge



Größenverhältnisse nicht realistisch!

Schematischer Entwicklungsablauf

- 
- Spezifikation der Anforderungen:
Was soll gemacht werden?
 - Design:
Wie wird die Aufgabe aufgeteilt?
Welche Bausteine werden benötigt?
 - Implementierung:
Technische Konstruktion der Bausteine
 - Test:
Passt die Lösung zur Anforderung?



Auf Ebene der
Architektur + auf Ebenen
darunter
(Objektorientierte
Programmierung:
Klassen)

Design bedeutet *Zerlegung*

Nach:

- Fachmodelle
- Daten
- Funktionen
- Abläufen (Workflows)

Design bedeutet

Formung der Einzelteile

- So einfach wie möglich
 - KISS – Keep it simple and stupid
 - Angemessen komplex
- Trennung nach Verantwortlichkeit
 - Module, Komponenten, Schichten
 - Technik und Fachlichkeit trennen
- Beziehungen herstellen
- Schnittstellen einziehen

Typische Design-Ziele

- Wartbare Software (Maintainability)
- Wiederverwendbarkeit (Reusability)
- Erweiterbarkeit (Extensibility)
- Sicherheit (Security)
- Benutzbarkeit (Usability, auch UX genannt)
- Kompatibilität
- Skalierbarkeit (Scalability)
- ...

OO-Design: Zentrale Aktivitäten

- Identifizieren von **Objekten** und **Klassen**
- Festlegen des **Verhaltens** der Objekte und Klassen
- Identifizieren von **Beziehungen** zwischen den Klassen
- Festlegen der **Schnittstellen** zwischen den Klassen

Richtige Wahl der Klassen ist wichtigster Schritt zu gutem objektorientierten Entwurf

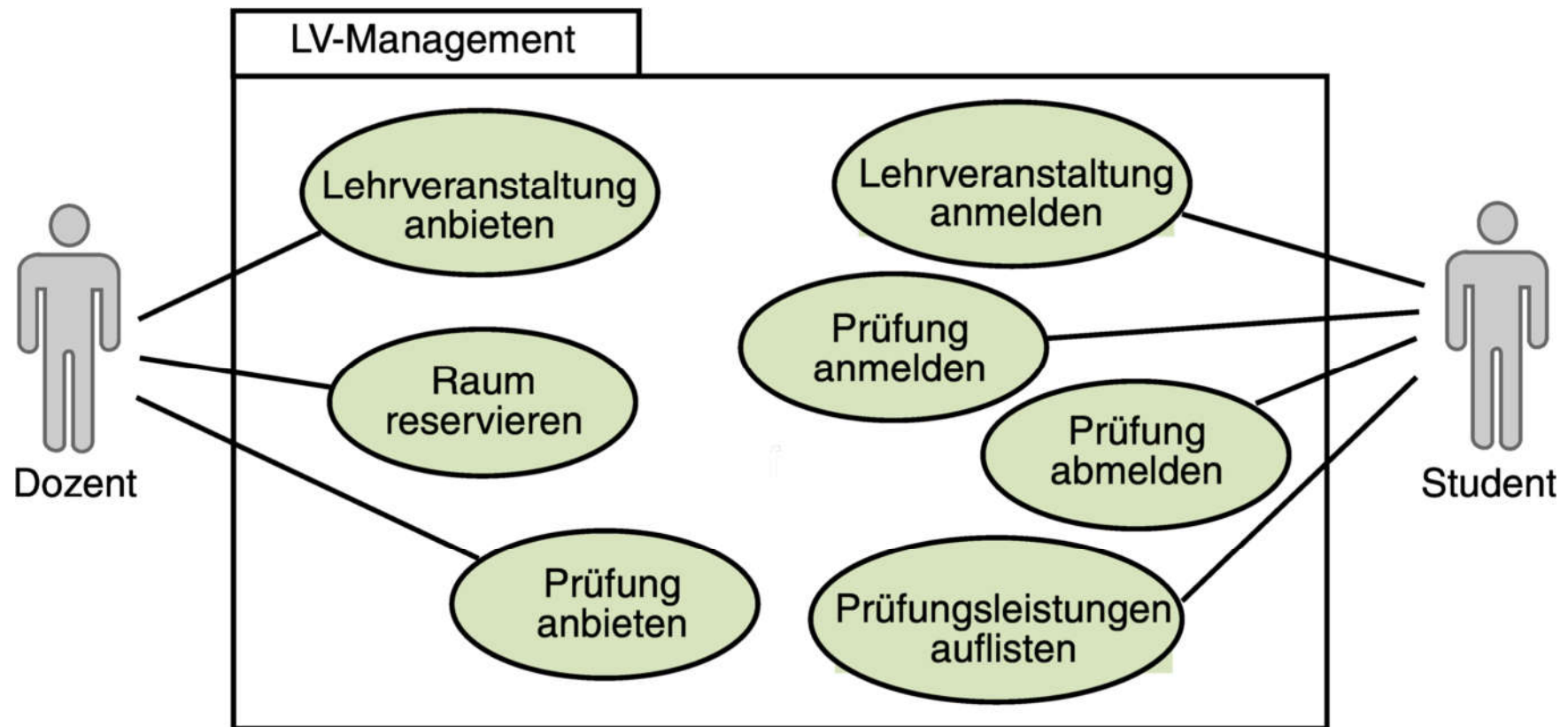
OO-Design:

Wie findet man Klassen?

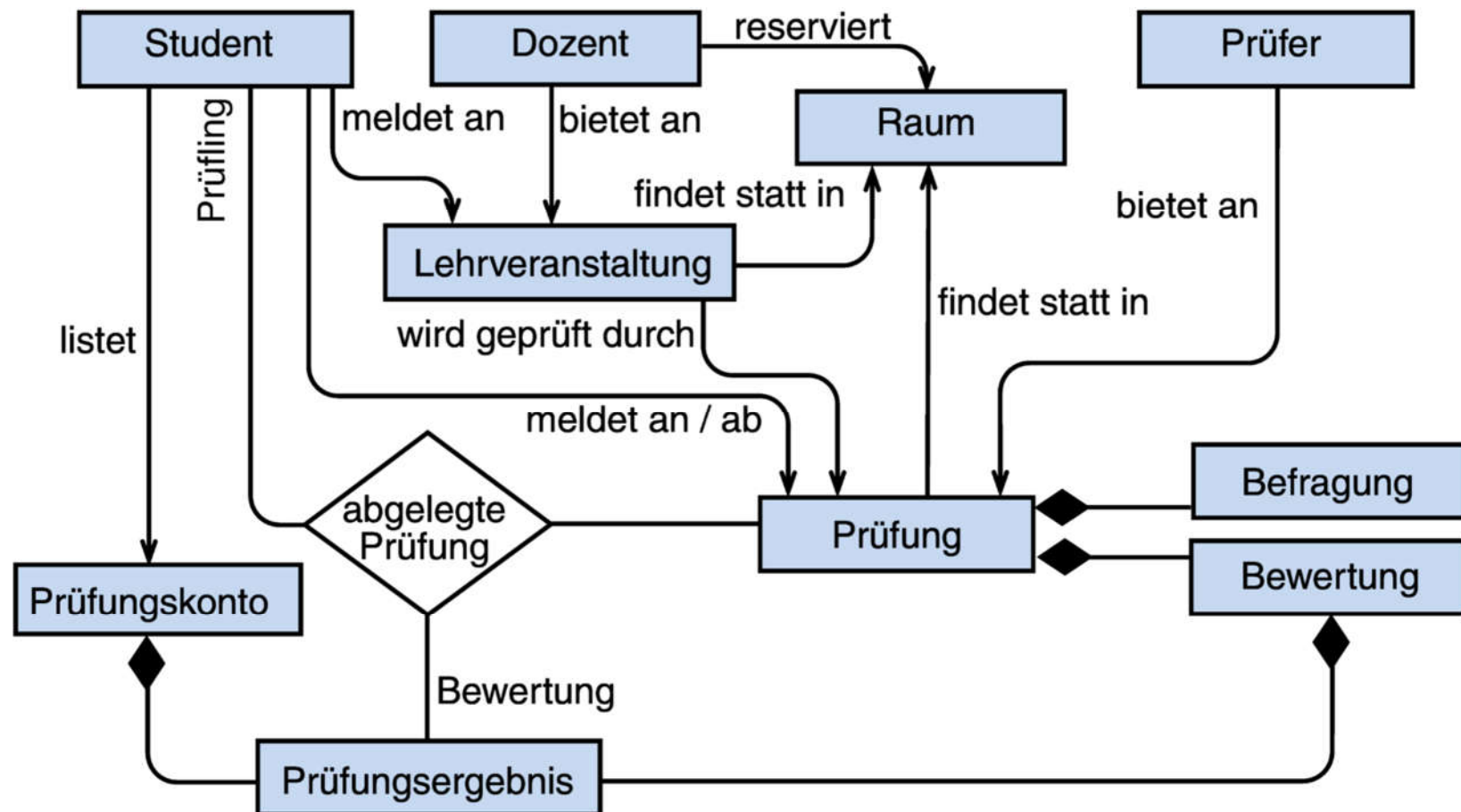
Im Anwendungsbereich wird die **Ist-Analyse** durchgeführt und das fachliche Modell entwickelt. Es besteht im Kern aus den **fachlichen Abläufen** und den **relevanten Gegenständen**.

- Die fachlichen Abläufe kann man in **Use Cases** modellieren.
- Die Gegenstände werden durch ein **objektorientiertes Begriffsmodell** beschrieben.

Beispiel: CAMPUS-System



Beispiel: CAMPUS-System



Gefahr: Beziehungen sind Abhängigkeiten

Software degeneriert im Laufe der Zeit:

„Verfaultes Design“ (Robert C. Martin)

Warum?

- Änderungen und Weiterentwicklungen kommen dazu
- Ursprüngliches Design hat die Weiterentwicklung nicht vorgesehen

Wie erkennt man verfaultes Design?

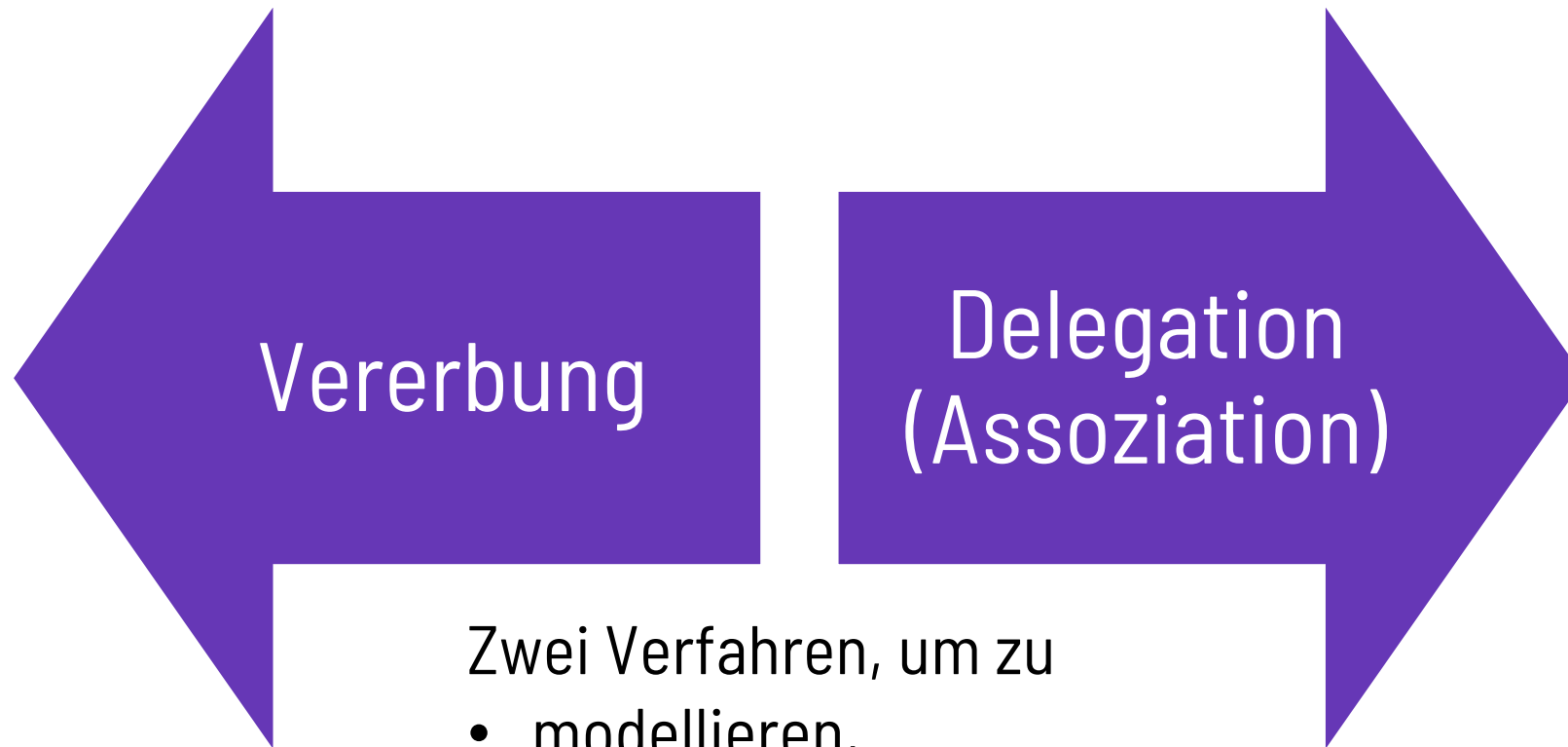
- Starrheit
 - Änderung und Austauschbarkeit schwierig
- Zerbrechlichkeit
 - Kleine Änderung zerstört Funktionalität in anderen Komponenten $\Rightarrow$ Weiterentwicklung schwierig!
- Schlechte Wiederverwendbarkeit
 - durch hohe Abhängigkeiten zu anderen Systemen
 - Bedeutet auch: Schlechte Testbarkeit!

Wichtiges Entwurfsziel: Steuerung von Koppelung und Kohäsion

Prinzip:

- Was **zusammen** gehört, soll und darf auch **eng** gekoppelt sein.
- Was **nicht** zusammen gehört, soll **lose** gekoppelt sein.

Grundlegende Beziehungen im objektorientierten Entwurf



Zwei Verfahren, um zu

- modellieren,
- Klassen zu erweitern,
- Funktionalität wieder zu verwenden

Grundlagen

Vererbung und Delegation (Assoziation)

- **Vererbung**: Schnittstellen- und Implementationsvererbung mit „ist-ein“-Beziehung
 - **White-Box** basierte Wiederbenutzung durch Bildung von Unterklassen
 - Definition zur **Kompilier-Zeit**
 - **Einfache** Benutzung
- Probleme mit Vererbung:
 - Keine dynamische Veränderbarkeit von Objektstruktur und -verhalten, da Vererbung zur Kompilier-Zeit definiert wird
 - Veränderungen der Basisklasse propagieren sich automatisch an Unterklassen weiter
 - Etwaige komplexe verhaltensbasierte bzw. implementierungsbasierte Abhängigkeiten zwischen Basis- und Unterklassen (siehe Open-Closed-Principle)

Grundlagen

Vererbung und Delegation (Assoziation)

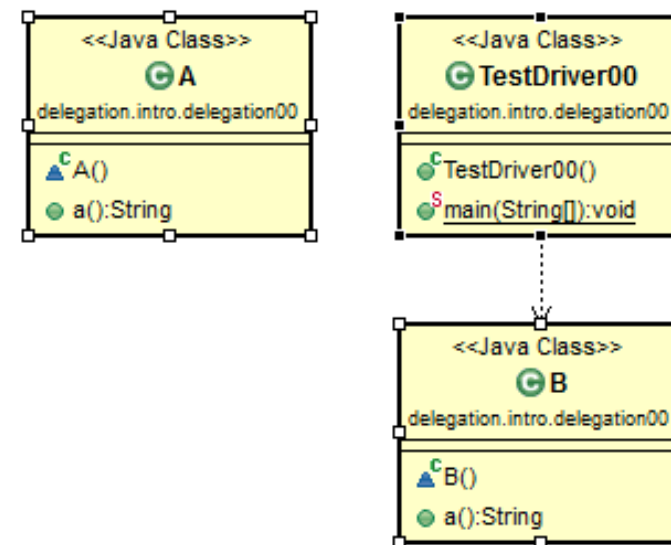
- **Delegation**: Operationen auf einem Objekt werden an ein anderes weitergereicht
 - **Black-Box** basierte Wiederverwendung
 - Definition zur Laufzeit ermöglicht die Komposition unterschiedlichen Verhaltens eines Objekts zur **Laufzeit** und erhöht somit die Flexibilität
 - Probleme mit Delegation:
 - Zusätzliche (logische) Indirektion kann zu **Leistungseinbußen** führen
 - **Anspruchsvollere** Benutzung
- Delegation als Ersatz für Vererbung möglich!

Nutzung von Vererbung und Delegation im Design

Beispiel

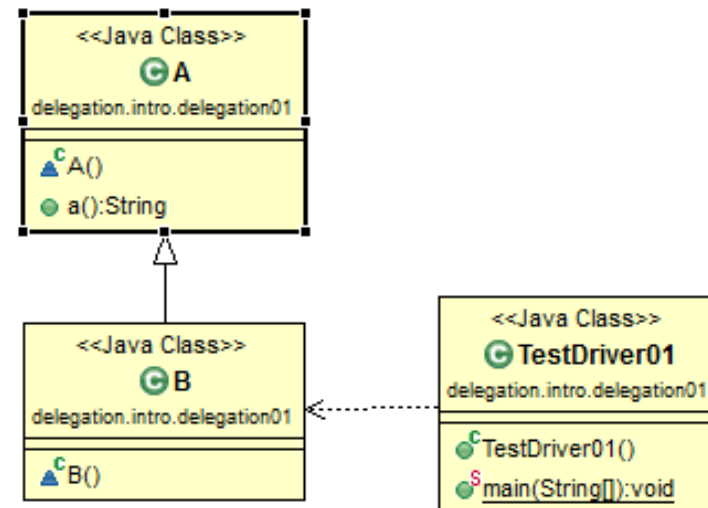
Klasse B soll Funktionalität von A mit a () zur Verfügung stellen!

```
1 package delegation.intro.delegation00;
2
3 public class TestDriver00 {
4     public static void main(String[] args) {
5         B b = new B();
6         System.out.println(b.a());
7     }
8 }
9 class A {
10     public String a() { return ("a"); }
11 }
12
13 class B {
14     public String a() { return ("a"); }
15 }
16
17
```



Nutzung von Vererbung und Delegation im Design Beispiel

```
1 package delegation.intro.delegation01;
2
3 public class TestDriver01 {
4     public static void main(String[] args) {
5         B b = new B();
6         System.out.println(b.a());
7     }
8 }
9 class A {
10     public String a() { return ("a"); }
11 }
12
13 class B extends A { }
```



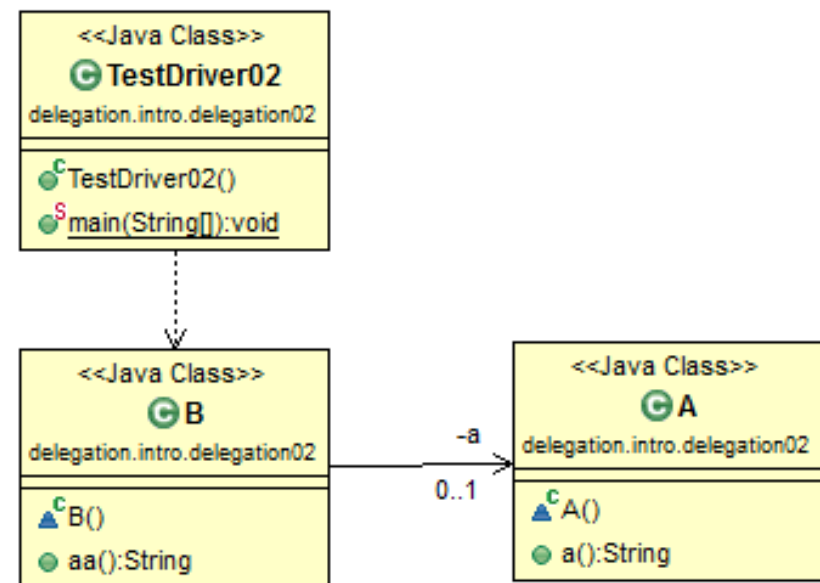
Eine Möglichkeit, theoretisch fertig.
Bewertung?

*Klasse B soll Funktionalität von A mit
a() zur Verfügung stellen!*

Nutzung von Vererbung und Delegation im Design

Beispiel

```
1 package delegation.intro.delegation02;
2
3 public class TestDriver02 {
4     public static void main(String[] args) {
5         B b = new B();
6         System.out.println(b.aa());
7     }
8 }
9 class A {
10     public String a() { return ("a"); }
11 }
12
13 class B {
14     private A a = new A();
15
16     public String aa() { return(a.a()); }
17 }
```



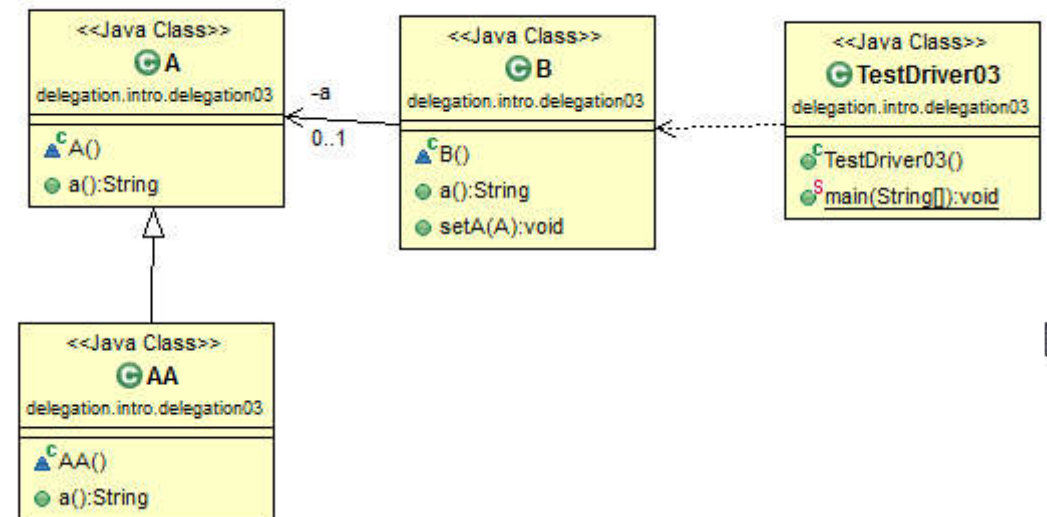
Wir probieren im Vergleich einen weiteren Ansatz aus....

Klasse B soll Funktionalität von A mit a() zur Verfügung stellen!

Nutzung von Vererbung und Delegation im Design

Beispiel

```
1 package delegation.intro.delegation03;
2
3 public class TestDriver03 {
4     public static void main(String[] args) {
5         B b = new B();
6         System.out.println(b.a());
7         b.setA(new AA());
8         System.out.println(b.a());
9     }
10 }
11 class A {
12     public String a() { return ("a"); }
13 }
14
15 class B {
16     private A a = new A();
17
18     public String a() { return(a.a()); }
19     public void setA(A a) { this.a = a; }
20 }
21
22 class AA extends A{
23     public String a() { return("aa"); }
24 }
```



Klasse B soll Funktionalität von A mit a() zur Verfügung stellen!

Nutzung von Vererbung und Delegation im Design

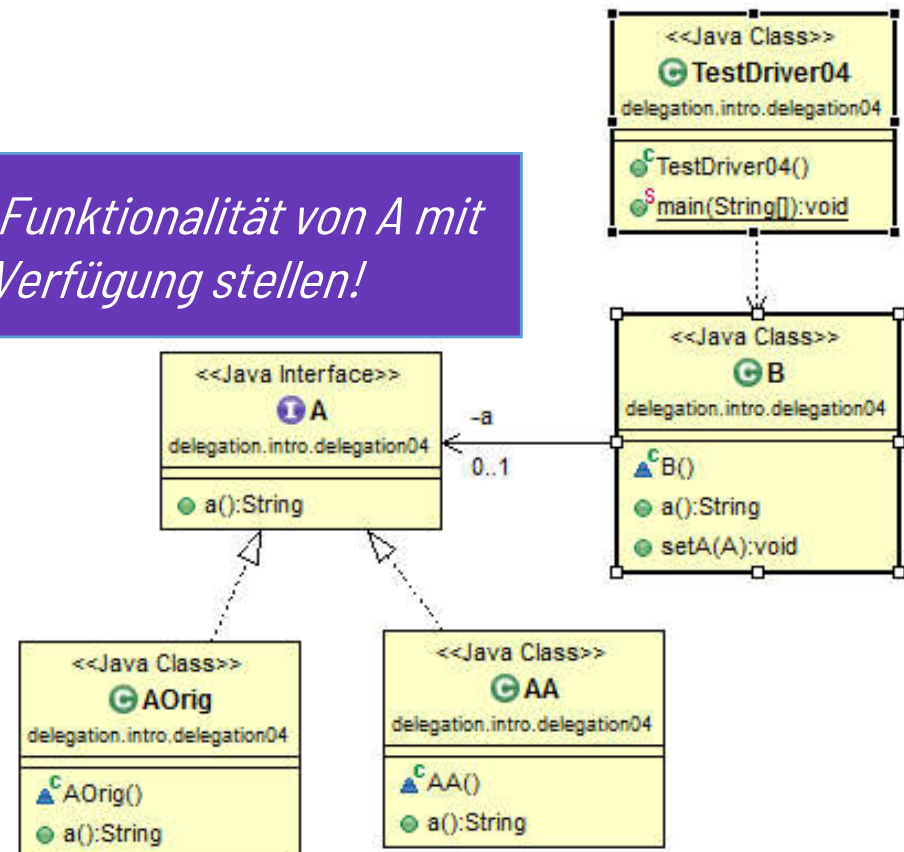
Beispiel

```

1 package delegation.intro.delegation04;
2
3 public class TestDriver04 {
4     public static void main(String[] args) {
5         B b = new B();
6         System.out.println(b.a());
7         b.setA(new AA());
8         System.out.println(b.a());
9     }
10 }
11 interface A {
12     public String a();
13 }
14
15 class AOrig implements A {
16     public String a() { return ("a"); }
17 }
18
19 class B {
20     private A a = new AOrig();
21     public String a() { return(a.a()); }
22     public void setA(A a) { this.a = a; }
23 }
24
25 class AA implements A {
26     public String a() { return("aa"); }
27 }

```

Klasse B soll Funktionalität von A mit a() zur Verfügung stellen!



Wichtige Prinzipien im (Objektorientierten) Design

(SRP) Single Responsibility Principle
(Prinzip der einen Verantwortlichkeit)

(OCP) Open Closed Principle
(Offen-Geschlossenheits-Prinzip)

(LSP) Liskov Substitution Principle
(Liskovsches Prinzip der Substituierbarkeit)

(ISP) Interface Segregation Principle
(Schnittstellenaufteilungsprinzip)

(DIP) Dependency Inversion Principle
(Prinzip der Umkehrung von Abhängigkeiten)

SRP – Single Responsibility Principle

SRP



Dass es *möglich* ist
heißt nicht,
das man es auch
tun soll.

SOLID
Motivational
Posters by
Derick Bailey

<https://lostechies.com/derickbailey/2009/02/11/solid-development-principles-in-motivational-pictures/>
V23.1 Software Engineering | Prof. Dr. Christoph Andriessens

SRP – Single Responsibility Principle (nach R. Martin)

SRP

Bei einer Klasse sollte es nur eine mögliche Ursache für Veränderungen geben.

- Kann man sich für eine Klasse mehr als einen Ursache vorstellen, sie verändern zu müssen, dann hat sie mehr als eine Verantwortlichkeit.
- Mehr Ursachen für Veränderungen, gleichzeitig eng gekoppelte Verantwortlichkeiten bedeuten
 - Probleme, Aufwand, Kosten
 - Je mehr mögliche Ursachen, desto häufiger



„So nicht“-Beispiel SRP

SRP

```
public class Employee {  
    public Money calculatePay();  
    public void save();  
    public String reportHours();  
}
```

Dafür ist Finanzchef verantwortlich.

Dafür ist Technikchef verantwortlich.

Dafür ist Betriebsleiter verantwortlich.

- `calculatePay`: Algorithmen zur Gehaltsberechnung basierend auf Arbeitsvertrag, Status, Arbeitsstunden
- `save`: Speichert Objekt in Datenbank
- `reportHours`: Erzeugt String zur Aufnahme in Prüfbericht für Auditoren zur Überwachung von Arbeitsstunden und Bezahlung

Und Ursachen für Veränderungen?

SRP

```
public class Employee {  
    public Money calculatePay();  
    public void save();  
    public String reportHours();  
}
```

Dafür ist Finanzchef verantwortlich.

Dafür ist Technikchef verantwortlich.

Dafür ist Betriebsleiter
verantwortlich.

- Ursachen für Veränderungen von `calculatePay` kommen vom **Finanzchef** und seinen Mitarbeitern,
- für Veränderungen von `save` vom **Technikchef** (und seinen Mitarbeitern),
- für Veränderungen von `reportHours` vom **Betriebsleiter** (und seinen Mitarbeitern).



„So nicht“-Beispiel SRP

SRP

```
public class Employee {  
    public Money calculatePay();  
    public void save();  
    public String reportHours();  
}
```

Dafür ist Finanzchef verantwortlich.

Dafür ist Technikchef verantwortlich.

Dafür ist Betriebsleiter verantwortlich.

Employee:

- Jede Methode hat eine andere Ursachen-Quelle für Änderungen
- Änderungen an einer beliebigen Methode gefährden die anderen Methoden
- Klasse erfüllt das SRP nicht
- Änderungen sind risikoreich: Und nur unter Gefahr der Verärgerung einer Stakeholdergruppe möglich



Also wie löst man es?

SRP

Kann Teil einer Geschäftslogik-Komponente werden.

```
public class Employee {  
    public Money calculatePay() ...  
}
```

Kann Teil einer Reporting-Komponente werden.

```
public class EmployeeReporter {  
    public String reportHours(Employee e) ...  
}
```

Kann Teil einer Repositoriums-Komponente werden.

```
public class EmployeeRepository {  
    public void save(Employee e) ...  
}
```

OCP – Open Closed Principle (nach B. Meyer)

OCP

Komponenten sollten offen für Erweiterung sein, geschlossen für Änderungen.

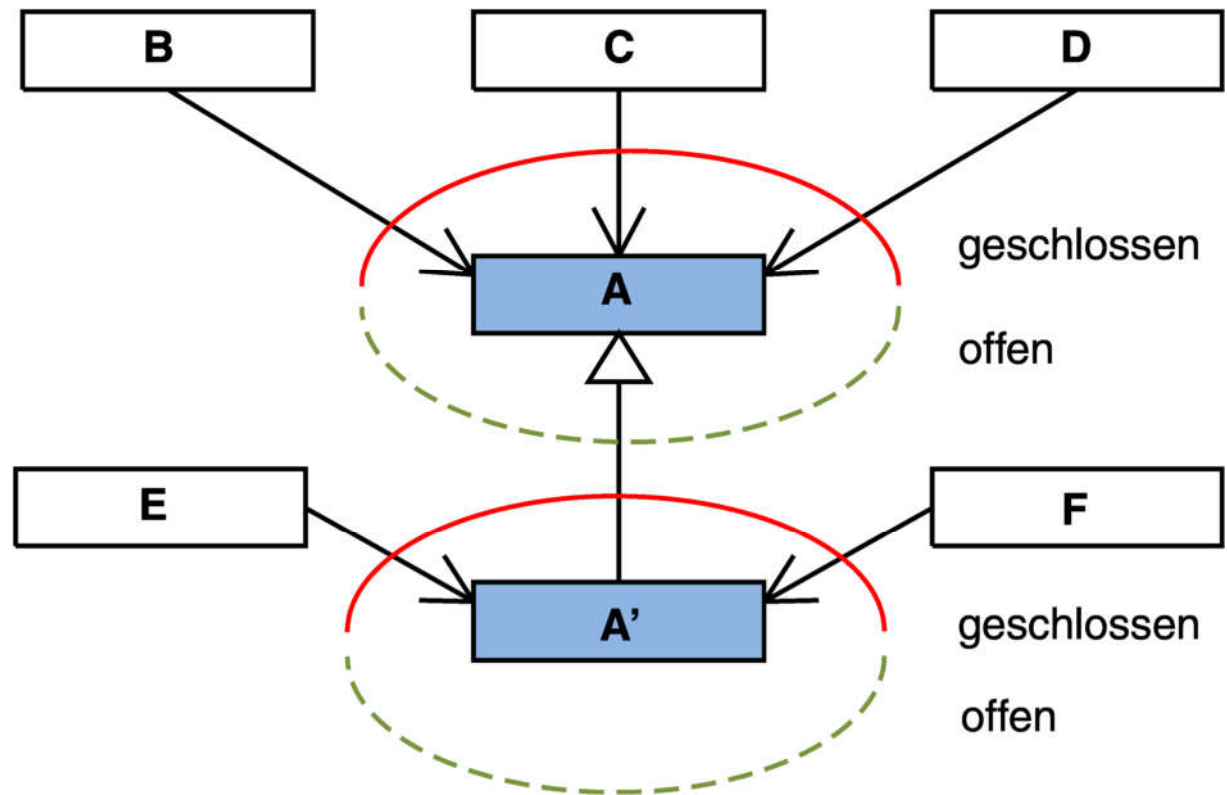
Komponenten und Klassen können verändert werden, ohne dass der zugehörige *Quellcode* geändert werden muss:

- Um das Verhalten eines Systems zu erweitern (z.B. durch eine Änderungsanforderung) wird *neuer* Quellcode erzeugt – der *bestehende* Quellcode bleibt unverändert
- Schlüsselprinzipien sind **Abstraktion** und **Polymorphismus**

Beispiel OCP

OCP

- B, C und D sind Kundenklassen von A.
- E und F sind neue Kundenklassen, die zusätzliche Funktionen benötigen.



Beispiel für OCP

User Story „Kontoverwaltung“

OCP

Als IT-Verantwortlicher einer Bank möchte ich eine Anwendung zur Verwaltung aller Konten meiner Kunden

Akzeptanzkriterien

- **Unterstützung von Sparkonto und Girokonto als Kontoarten**
- **Weitere Kontoarten werden in einem Jahr auf den Markt kommen und sollen vom System ohne grossen Aufwand unterstützt werden**

Beispiel für OCP

„Kontoverwaltung“: Erstellung ersten Entwurfs

OCP

- Welches sind die wesentlichen Klassen in ihrem objektorientierten Entwurf?
- In welchen Beziehungen stehen diese Klassen untereinander?
- Wie ist es mit dem Aspekt der Erweiterbarkeit!
- Entwurf dann als Klassendiagramm

Als IT-Verantwortlicher einer Bank möchte ich eine Anwendung zur Verwaltung aller Konten meiner Kunden

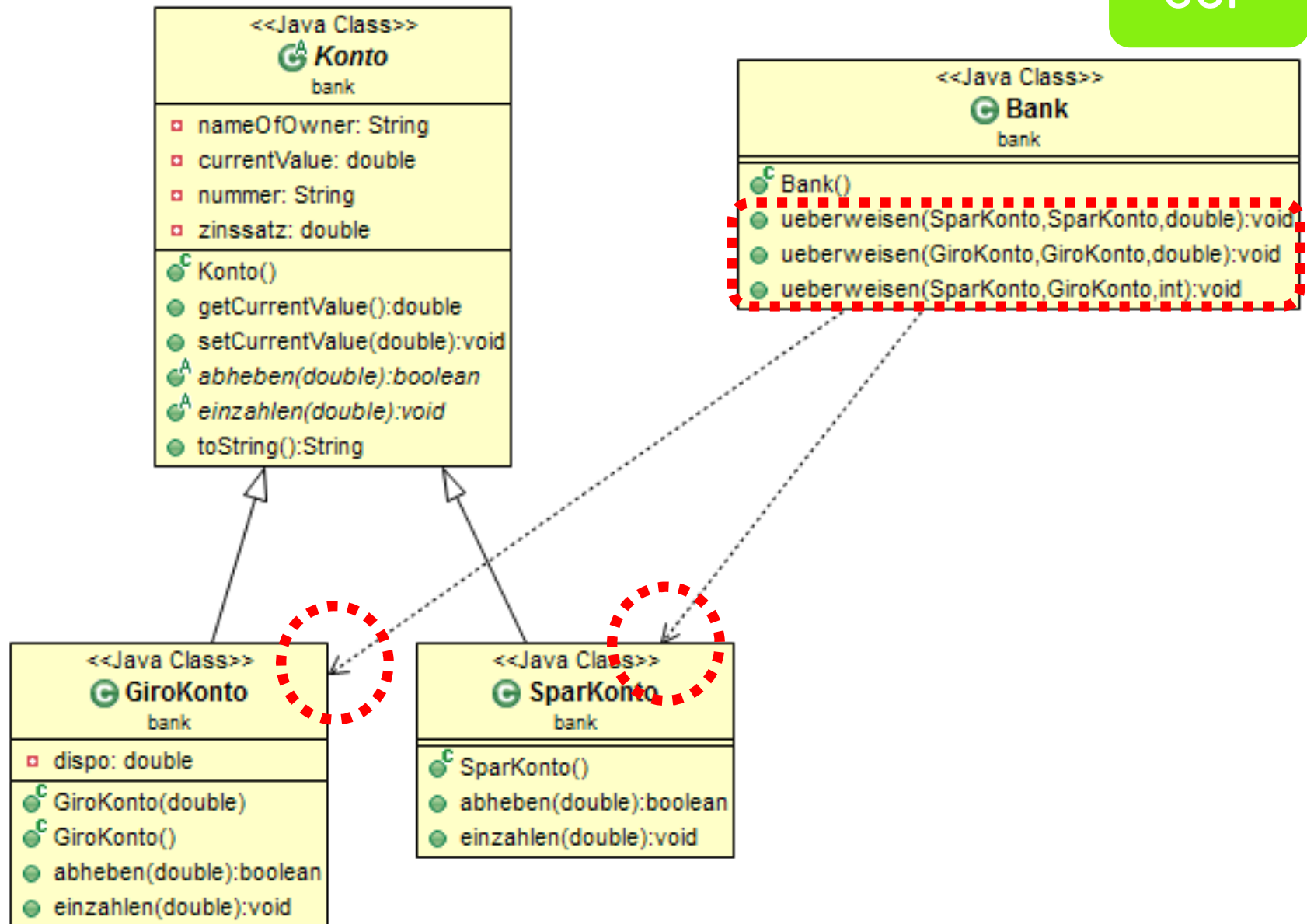
Akzeptanzkriterien

- Unterstützung von Sparkonto und Girokonto als Kontoarten
- Weitere Kontoarten werden in einem Jahr auf den Markt kommen und sollen vom System ohne grossen Aufwand unterstützt werden

Beispiel für OCP

Ergebnis mit verletztem OCP

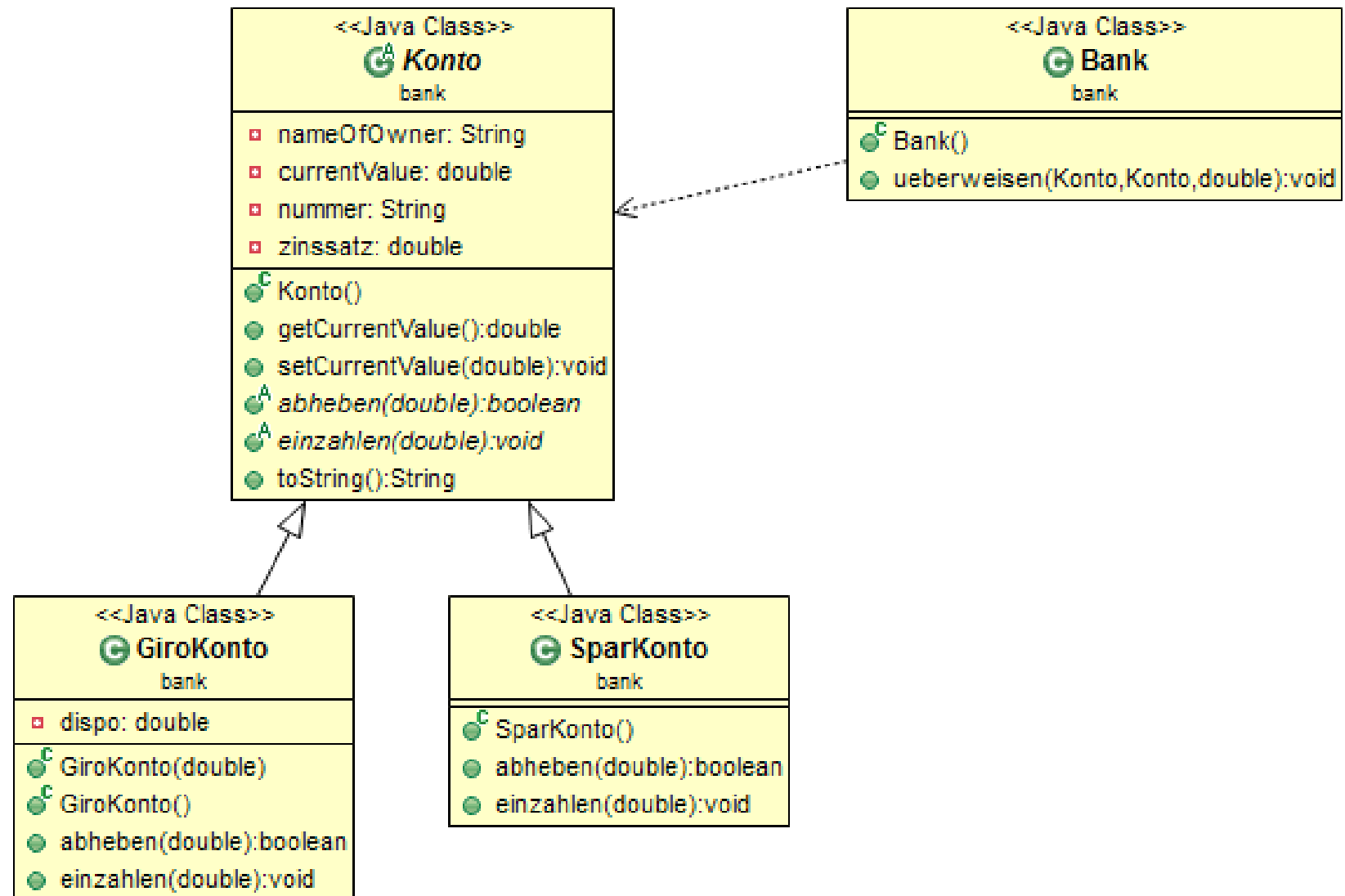
OCP



Beispiel für OCP

Ergebnis mit erfülltem OCP

OCP

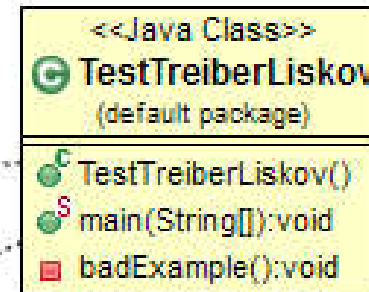
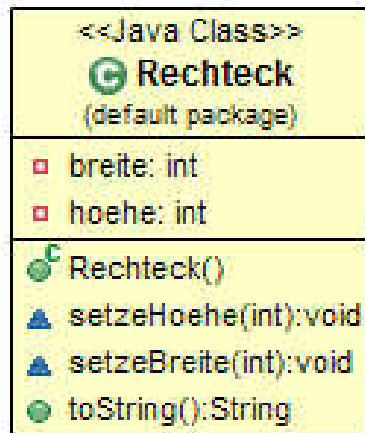


(LSP) Liskov Substitution Principle

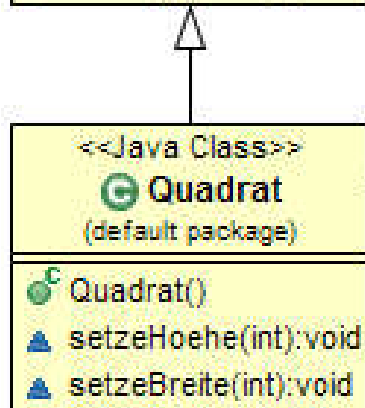
LSP

Klassen sollen in jedem Falle durch alle ihre abgeleiteten Klassen (Erben) ersetzbar sein.

- Bedeutet: „Unterklassen“ sollen nicht mehr erwarten (*an erfüllten Bedingungen*) und nicht weniger liefern (*an Funktionen*) als ihre „Oberklassen“
- Diese Ersetzbarkeit sorgt für Einhaltung der durch Klassen-Schnittstellen ausgedrückten „Verträge“
- Das erhöht Robustheit und Wartbarkeit von Systemen



LSP



```

Bank.java  Konto.java  konto.ucls  Rechteck.java  Quadrat.java  TestTreiberLiskov.java
7 public static void main(String[] args) {
8     TestTreiberLiskov liskov = new TestTreiberLiskov();
9     liskov.badExample();
10 }
11
12
13 private void badExample() {
14     Rechteck r = new Rechteck();
15     Quadrat q = new Quadrat();
16     Rechteck rq = new Quadrat();
17
18     r.setzeBreite(10);
19     q.setzeBreite(10);
20     rq.setzeBreite(10);
21
22     System.out.println(r + "\n" + q + "\n" + rq);
23 }
24
25 }
  
```

Problem | Javadoc | Declara | History | Console | Covera | Search | CPD Vi | Tasks | Servers | Call

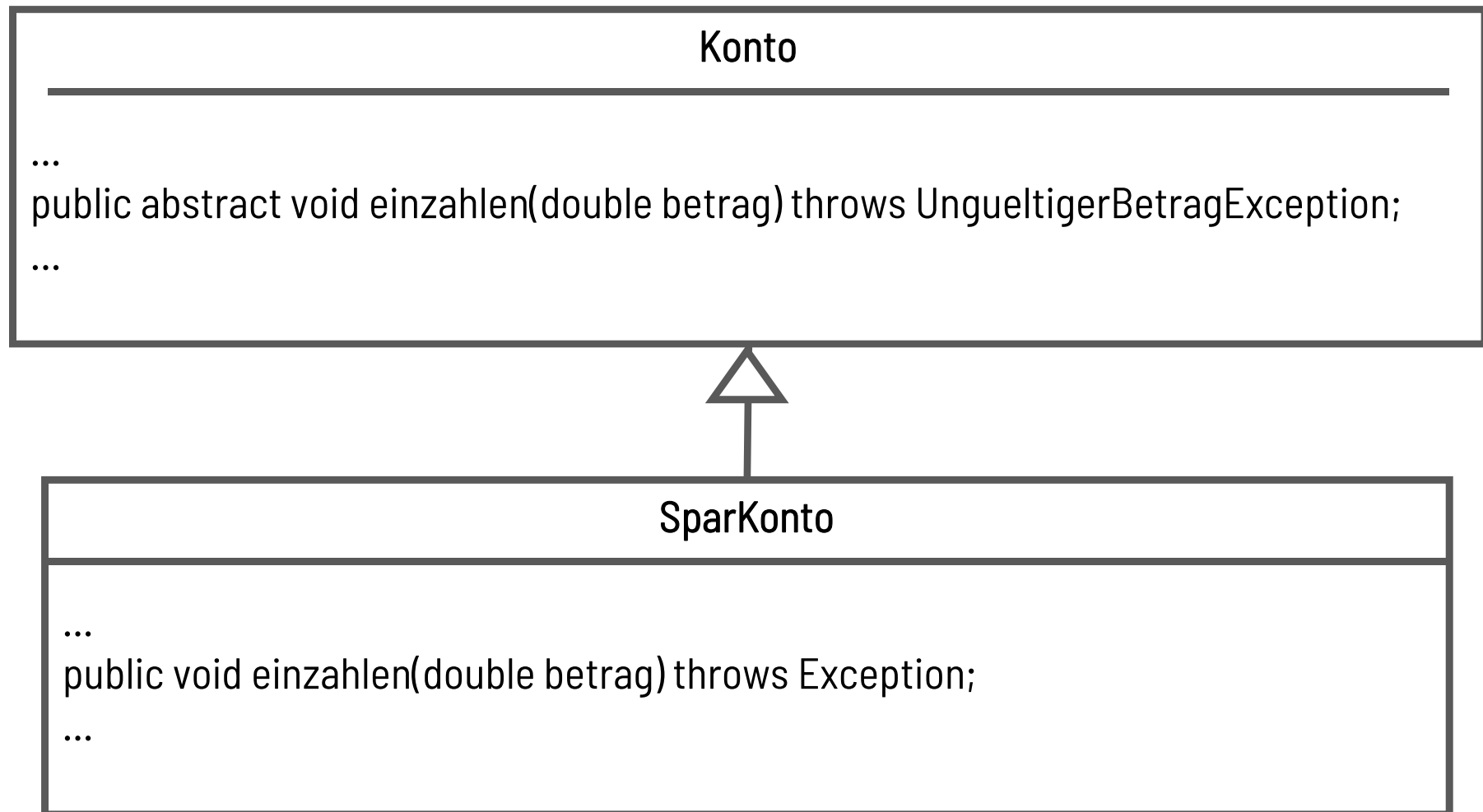
<terminated> TestTreiberLiskov [Java Application] C:\Program Files (x86)\Java\jre6\bin\javaw.exe (22.12.2011 19:48:31)

```

0/10
10/10
10/10
  
```

LSP klingt einfach und selbstverständlich, aber ist es das auch? Was geschieht in diesem Beispiel?

Praxisbeispiel für Verletzung LSP



Interface Segregation Principle – Abtrennung von Schnittstellen

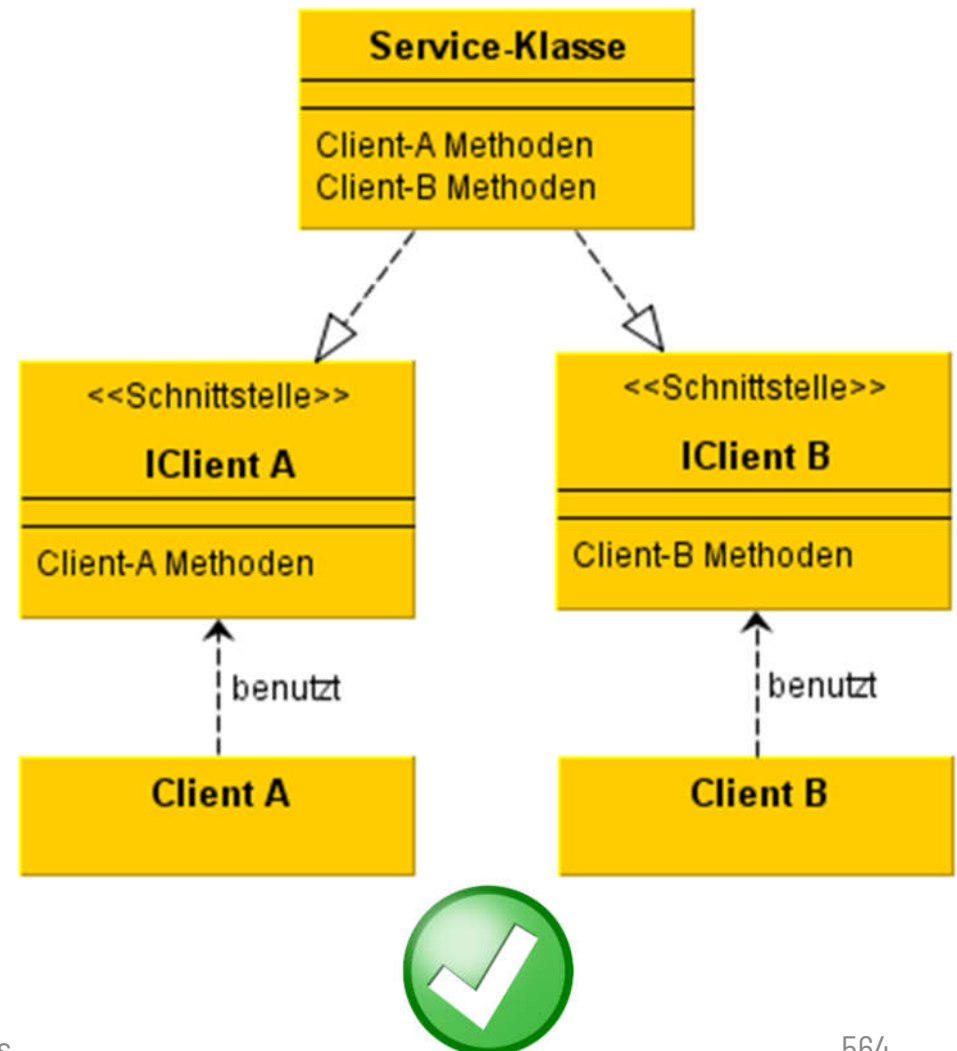
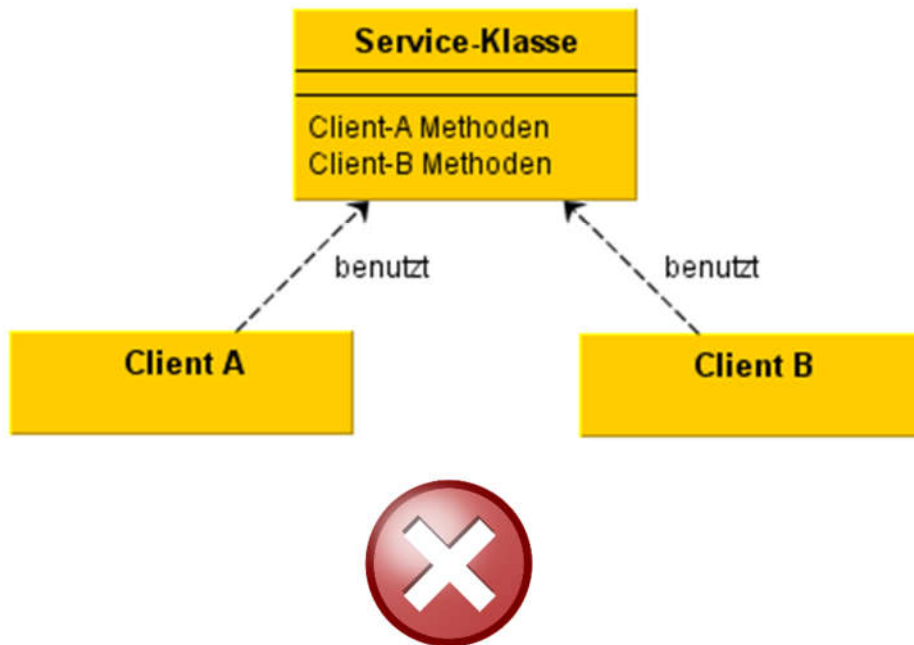
ISP

Mehrere clientspezifische Schnittstellen sind besser als eine große Universalschnittstelle.

- Modellierung verschiedener Verantwortungsbereiche einzelner Komponenten in jeweils einer Schnittstelle

Beispiel für ISP

ISP





Beispiel für ISP: So nicht, ...

ISP

```
public class ServiceKlasse {  
    public methodeA1();  
    public methodeA2();  
    public methodeA3();  
    public methodeB1();  
    public methodeB2();  
    public methodeB3();  
    ...  
}
```

```
public class ClientA {  
    ServiceKlasse sk;  
    ...  
}
```

```
public class ClientB {  
    ServiceKlasse sk;  
    ...  
}
```



Beispiel für ISP: ...sondern so!

ISP

```
public interface ClientAInterface {  
    public methodeA1();  
    public methodeA2();  
    public methodeA3();  
    ...  
}
```

```
public class ClientA {  
    ClientAInterface sk;  
    ...  
}
```

```
public interface ClientBInterface {  
    public methodeB1();  
    public methodeB2();  
    public methodeB3();  
    ...  
}
```

```
public class ClientB {  
    ClientBInterface sk;  
    ...  
}
```

```
public class ServiceKlasse  
    implements ClientAInterface,  
    ClientBInterface {  
    ...  
}
```

(DIP) Dependency Inversion Principle

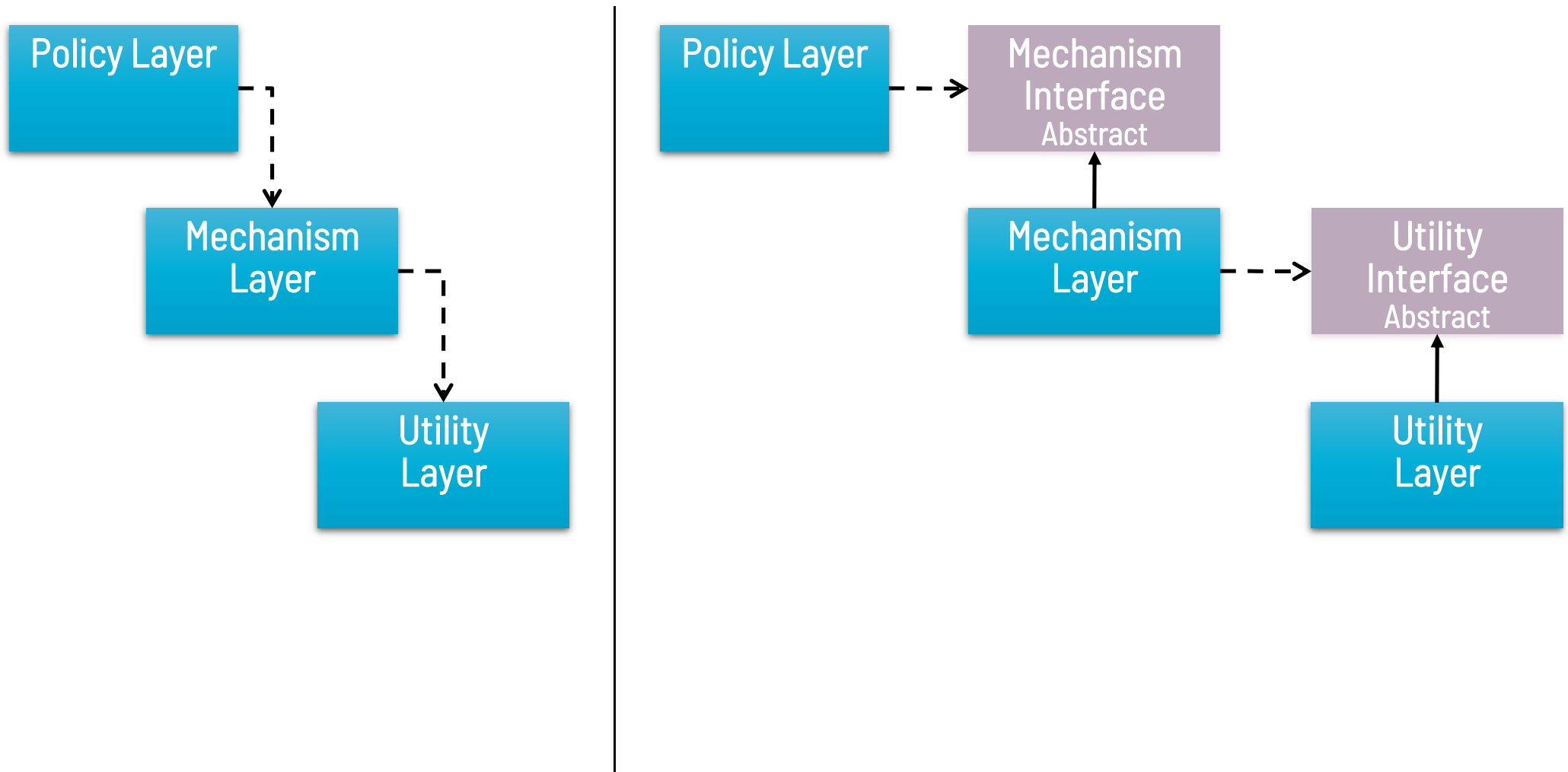
DIP

Erlauben Sie Abhängigkeiten nur von **Abstraktionen**, nicht von konkreten Implementierungen.

- Schlüssel für erweiterbare und flexible Architekturen
- Ansonsten besteht Gefahr schlechten Designs
 - Starrheit
 - System ist nur schwer zu ändern, da jede Änderung sehr viele andere Teile des Systems betrifft
 - Fragilität
 - Bei Änderungen „brechen“ nicht erwartete, andere Teile des Systems auseinander
 - Trägheit
 - Teile des Systems sind nur sehr schwer in anderen Kontexten wiederverwendbar

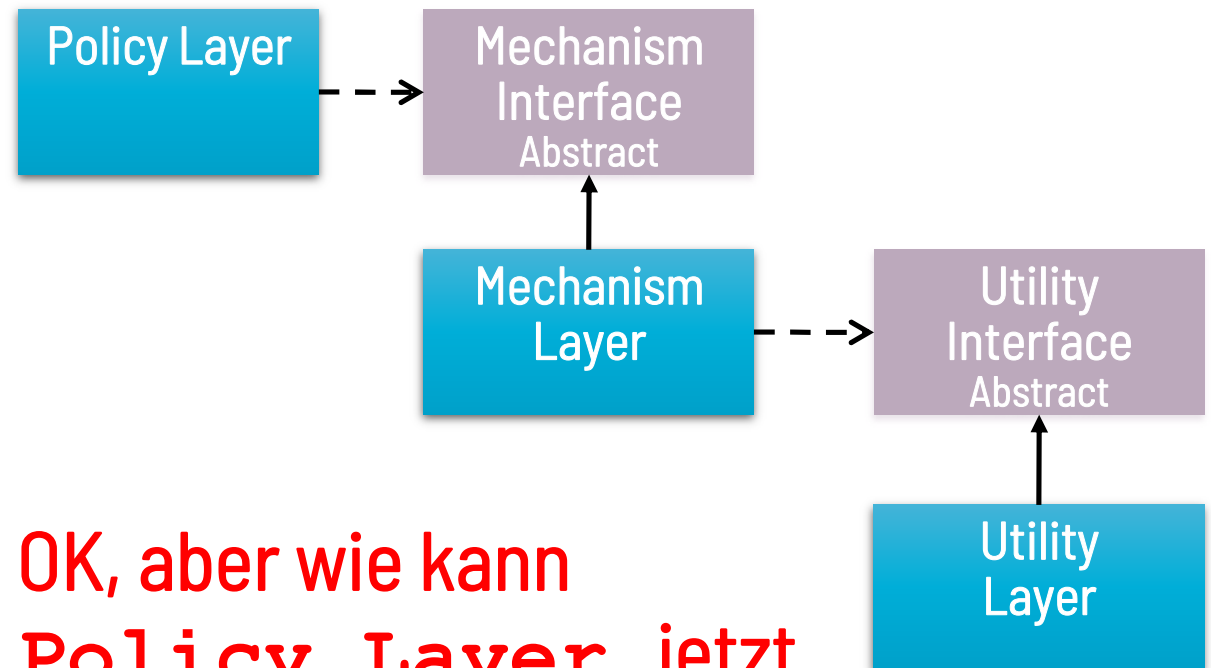
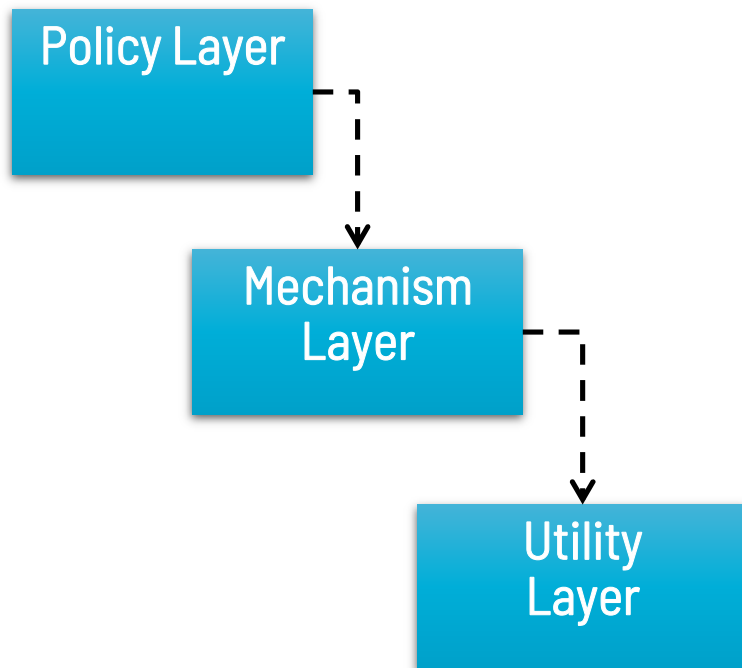
Allgemeines Beispiel für DIP

DIP



Allgemeines Beispiel für DIP

DIP



OK, aber wie kann
Policy Layer jetzt
auf einen konkreten
Mechanism Layer
zugreifen?

ISP-Beispiel erweitert um DIP

DIP

```
public interface ClientAInterface {  
    ...  
}  
public interface ClientBInterface {  
    ...  
}  
...  
public class ServiceKlasse  
{  
    implements ClientAInterface, ClientBInterface {  
        ...  
    }  
}
```

```
public class ClientA {  
    ClientAInterface sk;  
  
    public ClientA(ClientAInterface c)  
    {  
        this.sk = c;  
    }  
}
```

ClientB:
Genauso, nur mit
ClientBInterface

Verwendung von ClientA:

```
ClientA c = new ClientA(new ServiceKlasse());
```

DIP: Was bewirkt es?

Auflösung der Abhängigkeit durch

- Schnittstellen
- externe Erzeugung
- extern bestimmte Komposition



Als wissenschaftlicher Assistent möchte ich bei Twitter und bei Amazon nach Informationen suchen, damit ich schnell meine Rechercharbeiten abschliessen kann

DIP

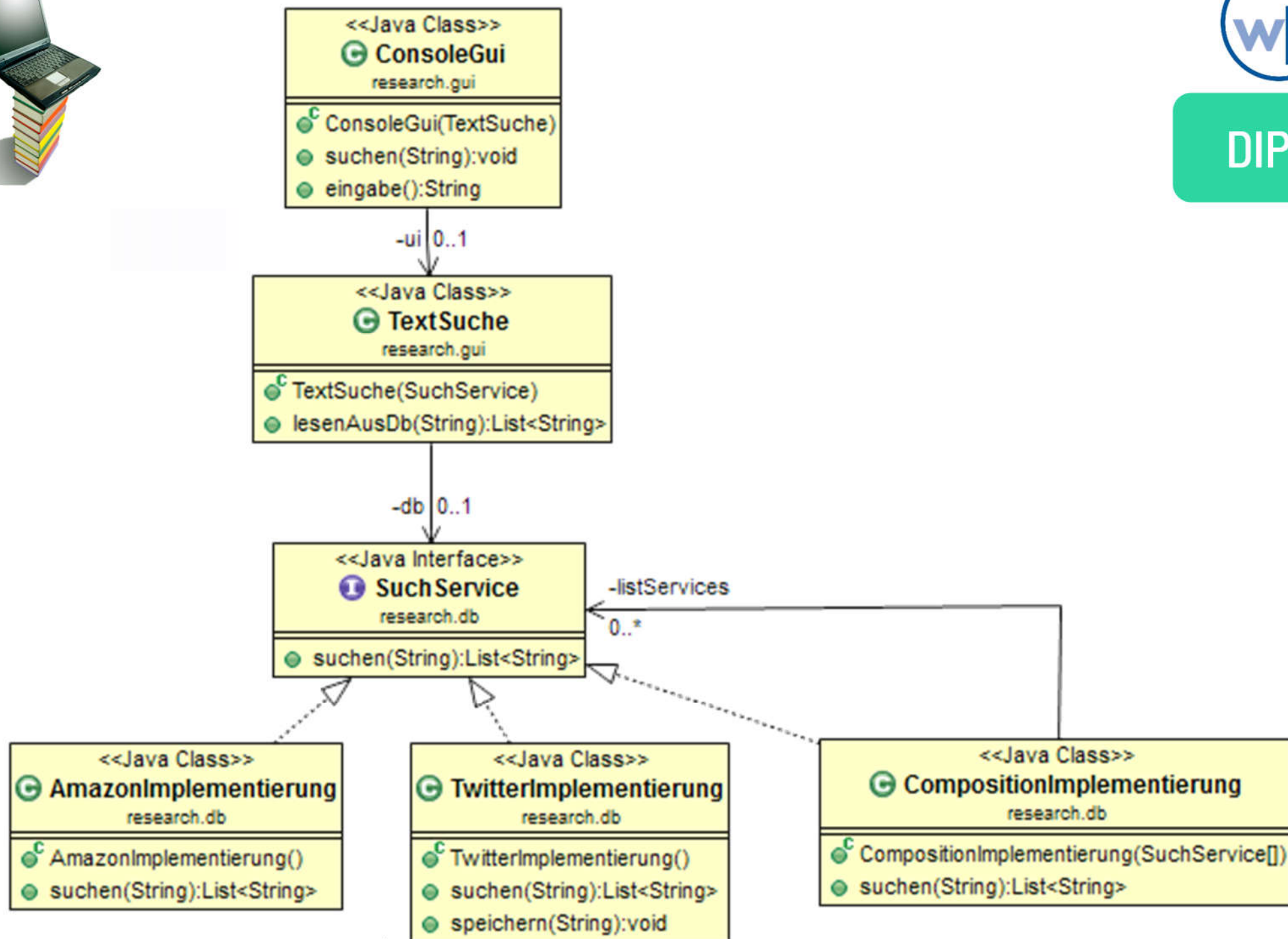
- Aufteilung nach Prinzip „Separation of Concerns“:
 - GUI: Zunächst einfache Lösung auf der Console
 - Such-Service
 - Zunächst: Anbindung an Twitter und Amazon
 - Später: an Google

Sie arbeiten in einem Team von 3 Personen an der Umsetzung dieser Anwendung.

- Wie gehen Sie am besten vor, damit Sie effektiv ihr Ziel erreichen und effizient die vorhandenen Ressourcen einsetzen?
- Achten Sie auf die Erweiterbarkeit ihres Entwurfs bzw. ihrer Architektur!



DIP



SOLIDe Software

SOLID ist eine Anwendung der Software Engineering-Prinzipien auf den objektorientierten Entwurf

(SRP) **Single** Responsibility Principle
(Prinzip der einen Verantwortlichkeit)

S

(OCP) **Open** Closed Principle
(Offen-Geschlossenheits-Prinzip)

O

(LSP) **Liskov** Substitution Principle
(Liskovsches Prinzip der Substituierbarkeit)

L

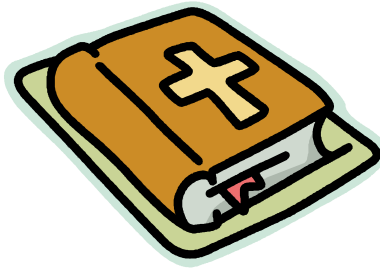
(ISP) **Interface** Segregation Principle
(Schnittstellenaufteilungsprinzip)

I

(DIP) **Dependency** Inversion Principle
(Prinzip der Umkehrung von Abhängigkeiten)

D

Heuristiken



- Regeln wie SOLID sind ein Beispiel für Heuristiken
- Heuristiken allgemein sind (unbewiesene) Regeln oder Hypothesen zum Vorgehen oder zur Erkenntnisgewinnung, als Gebote oder Verbote formuliert

Im Software-Entwurf:

Heuristiken sind Konkretisierungen von (Entwurfs- oder Software Engineering-)prinzipien in speziellen Kontexten

Zusammenfassung

- Wir haben die grundlegenden SOLID-Entwurfsprinzipien kennengelernt (und ausprobiert), die Prinzipien des Software Engineering in eine für den objektorientierten Entwurf nützliche Form kleiden
- Heuristiken konkretisieren Prinzipien weiter in Regeln für den Entwurf

